AUEC: A Contract Layer for Verifiable Hybrid AI Execution

Mohammed Messaoudene

***Abstract*—AI agents increasingly bridge remote models and local resources. Protocols such as MCP and A2A standardize discovery and invocation, while the authority semantics of hybrid execution remain largely product-specific: what data may be read, where an operation may run, which effects and egress are allowed, which budgets apply, how a result should be interpreted, and what evidence remains after execution. This report presents the AI Universal Execution Contract (AUEC), a transport-independent layer for bounded delegation. A remote planner submits a declarative execution graph; a host-controlled user agent derives the effective contract by intersecting that proposal with local capability, placement, information-flow, egress, and budget policies. AUEC separates epistemic status, representation, and information classification, and prevents an unvalidated claim from acquiring execution authority. Its deterministic U0 profile admits pure operations and emits canonical, hash-linked receipts. A Python reference runtime exposes MCP, A2A, and transport-neutral HTTP bindings. Pinned, unmodified official MCP runners pass 39/39 applicable checks for revision 2025-11-25, 22/22 for the tested 2026-07-28 release-candidate profile, and 85/85 for the associated tested draft suite. Four isolated negative controls each make the corresponding scenario fail and return it to success after restoration. Unmodified A2A testing remains incomplete (79/80 JSON-RPC and 76/78 HTTP+JSON). The artifact supports compatibility for the tested deterministic contract and causal sensitivity for four mechanisms; production security and end-to-end utility remain outside this evaluation.**

***Index Terms*—hybrid AI execution, protocol interoperability, host authority, capability security, epistemic non-escalation, information flow, provenance, tamper-evident receipts, MCP, A2A**

I. Introduction

REMOTE AI services can already ask local agents to read files, execute tests, or invoke applications. The unresolved problem is the portable meaning of those actions. Across providers and hosts, the same request should answer the same questions: Which resources may be accessed? Which effects are permitted? Where may each operation run? What resource limits apply? How should a result be interpreted? What evidence can another implementation verify after the run?

Most deployed agent loops keep these semantics inside product-specific code. A tool descriptor names an operation, a model selects it, and a host decides whether to invoke it. That arrangement enables rapid deployment, but it leaves authority, placement, and audit meaning outside the interoperability boundary. A successful tool call does not reveal whether the returned value is deterministic or probabilistic. A local audit log may be useful to one product without being portable to another. Transport success may also be mistaken for policy acceptance after the host has narrowed or rejected the proposed work.

The original design question behind the broader *AI Execution Web* (AIEW) was whether AI services need an analogue of the Web's portable client-side execution model. The analogy is useful only if authority remains local. JavaScript mattered because independently implemented user agents converged on a shared execution model, not merely because browsers consumed client CPU. AUEC carries that portability objective into AI delegation while replacing raw remote code with a bounded proposal. A host-controlled user agent validates the proposal, intersects it with local policy, and admits only the resulting contract.

AUEC occupies a semantic layer above existing transports and below application-specific planners and executors. MCP is the primary binding studied here; A2A provides an additional agent-to-agent path. Later profiles may use WebAssembly and WASI as execution substrates. The contract binds a proposed computation to host authority, information-flow rules, resource limits, result semantics, and portable evidence without redefining the transport or virtual machine.

Here, *verifiable* means that another implementation can check admission decisions, canonical result encodings, and receipt continuity under explicit assumptions. The term does not imply semantic truth, an uncompromised executor, or remote platform attestation.

A. Problem statement

A remote planner proposes work to a heterogeneous host. The planner may be correct, buggy, compromised, or influenced by untrusted context; the host owns the local data and resources and remains the final authority. The research problem is to define a provider-neutral contract that (i) describes a useful deterministic subset of hybrid work, (ii) cannot widen host authority or weaken data restrictions, (iii) distinguishes epistemic status from representation and confidentiality, and (iv) preserves the same contract semantics across transport bindings while exposing testable evidence.

B. Contributions

The report makes five bounded contributions.

1) **A hybrid-execution contract.** A declarative directed acyclic graph specifies operations, inputs, requested effects, admissible placements, result metadata, data classification, and resource budgets. Effective authority is derived by intersecting the remote proposal with host policy.

M. Messaoudene is a Maître de conférences B (MCB) at Belhadj Bouchaib University of Ain Temouchent, Ain Temouchent, Algeria (e-mail: mohammed.messaoudene@univ-temouchent.edu.dz; ORCID: 0009-0007-4665-2548). This affiliation identifies the author's academic appointment only and does not imply sponsorship, collaboration, or endorsement by the university.

2) **Epistemic non-escalation.** Deterministic facts, probabilistic claims, and hypotheses are separated from representation and confidentiality metadata. A value may influence consequential authority only after authorization-time validation and any action-bound consent required by the host.
3) **Portable tamper-evident evidence.** Canonical encodings and hash-linked node receipts bind a run to its manifest, inputs, outputs, placement, effect, and predecessor.
4) **A reference implementation over existing bindings.** A deterministic U0 profile is exposed through MCP, A2A, and HTTP without allowing transport metadata to widen host policy.
5) **A falsifiable conformance study.** Unchanged official MCP runners are paired with four causal negative controls. Incomplete A2A results remain visible rather than being relabeled as expected failures.

C. Contribution boundary and non-claims

The individual ingredients have substantial prior art: capability systems, policy intersection, resource budgets, information-flow lattices, cryptographic logs, portable execution, and agent protocols all predate this work. The contribution evaluated here is their composition into one transport-independent contract for a probabilistic remote proposer: host policy determines effective authority; epistemic validation constrains which values may authorize effects; and canonical receipts preserve execution evidence across bindings. The study does not establish production isolation, provider cost savings, semantic truth of model outputs, legal novelty, or adoption by a standards body.

D. Paper organization

Section II positions the work relative to portable execution, agent protocols, recent agent-governance systems, local–cloud partitioning, capability security, and provenance. Section III defines the roles, adversary, and requirements. Section IV presents the contract and its invariants. Section V describes the reference runtime and bindings. Sections VI and VII give the evaluation method and results. Sections VIII to X discuss security, implications, and limitations, and Section XI documents reproducibility.

II. Background and Related Work

A. Portable networked execution

The Web separated resource transfer, representation, and local execution, allowing independently developed implementations to converge on common behavior. Architectural constraints such as statelessness and uniform interfaces likewise made interoperation a system property rather than a library convention [1]. AUEC adopts this portability objective, but the browser threat model does not transfer unchanged: an AI planner may construct requests from untrusted documents and probabilistic inferences, so the local authority boundary must be explicit.

WebAssembly provides a portable low-level code format while leaving environmental access to the embedder [2], [3]. WASI 0.3 adds asynchronous component primitives [4]. These technologies define execution substrates. They do not specify the hybrid-work proposal, information-flow policy, epistemic status, or receipt semantics studied here. AUEC treats them as possible hosts for later profiles.

B. Tool and agent interoperability protocols

MCP standardizes discovery and invocation of tools, resources, and prompts, and its revisions are accompanied by conformance tooling [5], [6], [7]. At the evidence cutoff, the official release registry identified 2025-11-25 as the latest stable revision and 2026-07-28 as a release candidate whose specification remained in draft form. A2A standardizes discovery, messages, tasks, streaming, and artifacts between agents whose internals may remain opaque [8], [9]. Both protocols serve as bindings in this work.

Binding and contract remain separate. A generic A2A message must remain valid when the peer does not implement AUEC. An MCP exchange may succeed at the transport layer while host policy rejects the embedded execution proposal. The binding carries the proposal; the contract result records whether it was accepted, narrowed, completed, aborted, or left waiting for input.

C. Edge, split, and local–cloud AI execution

Partitioning computation between devices and servers is well established. Neurosurgeon schedules neural-network layers across mobile and cloud resources [10]; Edgent combines DNN partitioning with early exits [11]; survey work organizes split computing around latency, energy, bandwidth, and accuracy trade-offs [12]; and BottleNet++ studies compression at the partition boundary [13]. These systems optimize a particular model or workload. AUEC instead specifies how heterogeneous planners and hosts describe admissible work, hard policy constraints, and evidence.

Local-Splitter is close to the deployment motivation. Its 2026 preprint evaluates local routing, prompt compression, caching, and related tactics for coding agents, with workload-dependent reductions in cloud-token use [14]. Those measurements are prior work, not results for AUEC. They also show why savings and quality must be measured together. The present evaluation concerns protocol conformance and causal test sensitivity.

D. Capability security and information flow

Fail-safe defaults, complete mediation, and least privilege are longstanding principles of secure system design [15]. Information-flow lattices provide a compact representation of monotonic security classes [16]. AUEC applies these principles to remote proposals: no ambient interface is granted, and an output cannot be classified below the least upper bound of its inputs.

The four-level lattice used here is deliberately modest. It is not a complete mandatory-access-control model and does not establish noninterference. Its purpose is to make a portable no-downgrade rule and an egress ceiling testable in the deterministic base profile.

E. Provenance, tamper evidence, and attestation

PROV-O supplies a vocabulary for interoperable provenance [17]. Hash chaining and time-stamping have a long history in tamper-evident records [18]. RATS separates attesters, verifiers, evidence, and appraisal policy [19], while in-toto binds software supply-chain steps to signed metadata [20].

AUEC receipts occupy a narrower layer: they bind the declared steps and canonical results of one execution. A valid chain does not prove that the host was uncompromised, that a probabilistic claim is true, that the chain is fresh, or that no side channel exists. Signatures, durable replay protection, trusted time, and platform attestation belong to additional profiles.

F. Contemporary agent contracts, receipts, and delegation evidence

Recent systems address adjacent parts of this problem. NabaOS classifies statements by epistemic source and checks them against HMAC-signed tool receipts to detect hallucinated or misstated tool results [21]. Its decision is primarily a posteriori: the receipt helps a user assess whether a response is grounded. AUEC places a coarser epistemic distinction at the authorization boundary, before a value may influence a consequential local effect.

Agent Contracts formalizes multidimensional resource limits and conservation laws for hierarchical agent delegation [22]. AUEC uses a related narrowing principle at a different boundary: a remote planner proposes work to a host that independently controls capabilities, placement, egress, and budgets. Delegation-scoped observability binds execution events to a delegation context for cross-tool reconstruction [23]; capability-governance work binds tool identities to certificates and verifiable ledgers [24]; and receiver-attested receipts move evidence generation to the service that observes an action [25]. These systems strengthen grounding, attribution, or audit. They do not replace the host-computed execution contract studied here, and several provide stronger receipt trust models than the unsigned U0 chain.

G. Positioning

Table I locates the contribution. No component is claimed as novel in isolation. The distinctive design choice is to combine host-controlled policy intersection, placement and budget constraints, epistemic authorization, and portable tamper-evident receipts in one contract that can traverse existing agent protocols. Whether this composition is legally novel or broadly useful requires evidence beyond the present artifact.

III. System Model, Threats, and Requirements

A. Entities and trust boundary

We distinguish five logical roles (Figure 1).

- **Planner** P**:** a remote AI service or agent that proposes a manifest. P is not trusted with local authority.
- **User agent** U**:** the host-controlled decision point. It parses the proposal, intersects it with local policy, requests consent when required, and selects admissible placement.
- **Executor** X**:** a local, edge, or cloud substrate admitted by U for one node under an effective contract.
- **Binding** B**:** MCP, A2A, HTTP, or an offline envelope. B transports messages but does not determine authority.
- **Verifier** V**:** a party that checks canonical results, receipt continuity, and any additional signature or attestation profile.

The local policy is trusted relative to the remote proposal. The operating system, compiler, runtime, and hardware are not assumed infallible. A production deployment would require a more carefully delimited trusted computing base and independent evaluation.

B. Adversary model

The planner may be malicious, compromised, or influenced by prompt-injected content. It can attempt to widen capabilities, lower data classifications, amplify output, force an expensive placement, reuse stale consent, or present a model assertion as authority. A network attacker may alter messages. A tenant may replay a valid request with different input or policy. A verifier may receive a truncated, reordered, or equivocated receipt sequence. The implementation may contain parser, concurrency, persistence, or boundary errors.

The base profile does not address a compromised kernel, malicious toolchain, microarchitectural side channels, or semantic deception inside a probabilistic model. Those risks are outside the claims of this study.

C. Requirements

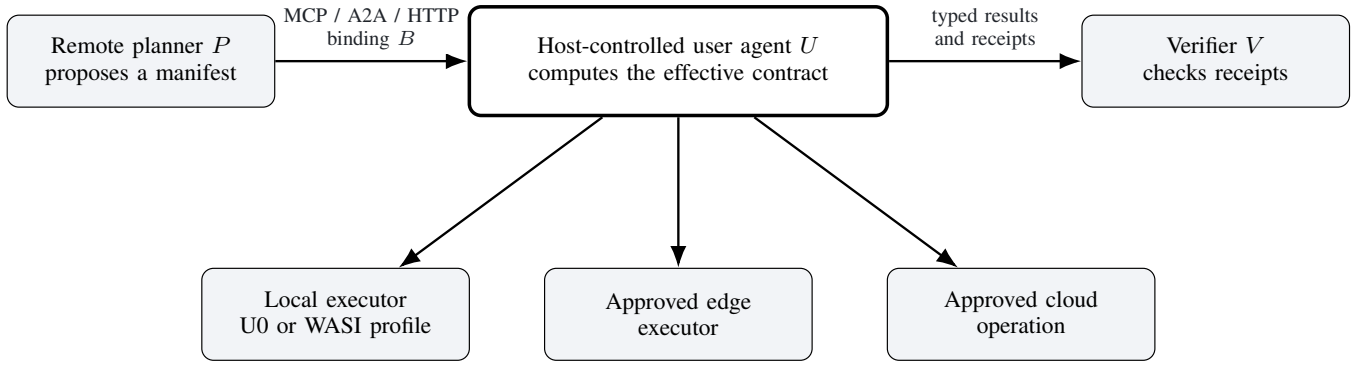
The design follows eight requirements.

- **REQ-1 - Host authority:** The planner proposes; the host may narrow or reject the proposal but cannot be induced to widen authority.
- **REQ-2 - Transport neutrality:** Semantically equivalent manifests have the same contract meaning across bindings.
- **REQ-3 - Least authority:** Filesystem, network, process, model, device, and accelerator interfaces are unavailable unless a profile and host policy grant them.
- **REQ-4 - Information monotonicity:** A node cannot downgrade the classification induced by its inputs, and the host's egress ceiling is a hard bound.
- **REQ-5 - Resource boundedness:** Planner budgets are upper bounds that host policy may reduce further.
- **REQ-6 - Epistemic non-escalation:** A `claim` or `hypothesis` cannot become action authority without an explicit verification transition and any consent required for the exact action.
- **REQ-7 - Tamper-evident execution evidence:** Deterministic runs produce canonical receipts for which modification of a complete chain is detectable under stated cryptographic assumptions.
- **REQ-8 - Binding compatibility:** Unnegotiated MCP and A2A behavior remains available.

These requirements are stronger than a successful tool invocation and substantially weaker than a production security certification.

TABLE I
POSITIONING RELATIVE TO ADJACENT MECHANISMS. ENTRIES DESCRIBE NORMATIVE COVERAGE OF THE LISTED CONCERN, NOT OVERALL SYSTEM CAPABILITY.

Mechanism	Tool/agent messaging	Portable local code	Placement and budget semantics	Epistemic authority rule	Portable execution receipts
MCP	Native	Not its role	Implementation-specific	Not normative	Not normative
A2A	Native	Not its role	Not normative	Not normative	Not normative
WebAssembly/WASI	Not its role	Native	Host-defined	Not its role	Not its role
Split-computing systems	Workload-specific	Model-specific	Native to optimizer	Not typical	Experimental logs
PROV-O/RATS/in-toto	Evidence/provenance	Not its role	Policy-specific	Attestation-dependent	Evidence structures
Recent agent-governance systems	System-specific	Varies	Partial or native	Partial	Native in some systems
AUEC (this work)	Through bindings	Profile-dependent	Normative	Normative	Normative



The binding transports a proposal; only U computes the effective contract.

Fig. 1. AUEC trust boundary and execution roles.

IV. AUEC DESIGN

A. Manifest as a proposal

An AUEC manifest is a versioned directed acyclic graph $G = (N, E)$ containing resources, global budgets, nodes, and exports. Each node $n \in N$ names an operation, inputs, requested effect, admissible placements, declared result metadata, and local budgets. Unknown normative fields are rejected unless a negotiated extension defines them. Ambiguity is therefore a protocol error rather than an implementation choice. In the accompanying draft specification, capitalized requirement keywords are interpreted according to BCP 14 [26], [27].

The host does not execute the manifest as ambient code. It parses the proposal, resolves references, validates the graph, and computes the effective contract. Let $C_p(n)$ denote the capabilities proposed for node n and $C_h(n)$ those permitted by host policy. The effective set is

$$C_{\text{eff}}(n) = C_p(n) \cap C_h(n). \tag{1}$$

Placement is narrowed in the same way:

$$P_{\text{eff}}(n) = P_p(n) \cap P_h(n), \qquad P_{\text{eff}}(n) \neq \varnothing. \tag{2}$$

If the intersection is empty, the node is rejected rather than silently redirected.

For every bounded resource q, the effective limit is

$$b^q_{\text{eff}}(n) = \min\{b^q_p(n), b^q_h(n)\}. \tag{3}$$

The base profile bounds manifest bytes, node count, wall time, and canonical output bytes. Later profiles may add memory, accelerator time, network volume, or energy; the present evaluation does not measure those quantities.

a) Policy-monotonicity invariant: Equations 1–3 make the effective contract no more permissive than either input policy. A planner request cannot create a capability absent from C_h, add a placement absent from P_h, or increase a host budget. This invariant assumes a trusted policy interpreter and extension processing that cannot bypass the intersection step.

B. Information classification and egress

The base information-classification lattice is

$$\mathsf{public} \sqsubset \mathsf{internal} \sqsubset \mathsf{confidential} \sqsubset \mathsf{secret}. \tag{4}$$

For a node n with input set $I(n)$, the declared output class must satisfy

$$\ell_{\text{out}}(n) \sqsupseteq \text{lub}\{\ell(x) \mid x \in I(n)\}, \tag{5}$$

where lub denotes the least upper bound in the lattice. The host egress ceiling $\ell^{\max}_h$ is independent of the manifest. An export is admitted only when $\ell_{\text{out}} \sqsubseteq \ell^{\max}_h$. The U0 profile prohibits export of secret data. This rule prevents explicit downgrading; it is not a noninterference theorem and does not account for timing or other side channels.

C. Epistemic status, representation, and authority

A result has three orthogonal dimensions:

$$\kappa(x) \in \{\texttt{fact}, \texttt{claim}, \texttt{hypothesis}\}, \tag{6}$$
$$\rho(x) \in \{\textsf{inline}, \textsf{artifact}\}, \tag{7}$$
$$\ell(x) \in \{\textsf{public}, \textsf{internal}, \textsf{confidential}, \textsf{secret}\}. \tag{8}$$

The epistemic kind κ describes how a value may be used as knowledge. A `fact` is deterministic under the declared operation semantics; it is not a claim of universal real-world truth. A `claim` is probabilistic, and a `hypothesis` is a candidate explanation. Representation distinguishes inline values from content-addressed artifacts, while confidentiality is expressed only by ℓ.

The compact U0 wire vocabulary predates this conceptual separation and uses one field for a subset of result tags. The normative model treats the three dimensions independently; a future schema revision should expose that separation while retaining a versioned compatibility path.

The authority rule is asymmetric:

$$\begin{aligned}\mathrm{Auth}_U(x,a) \Rightarrow\ & \kappa(x) = \texttt{fact} \\ & \wedge \mathrm{validated}_U(x,a) \\ & \wedge \mathrm{consent}_U(a).\end{aligned} \tag{9}$$

The predicate $\mathrm{validated}_U(x,a)$ is an authorization-time check performed by the host. It binds the value's declared provenance and operation evidence to the exact action a; it is distinct from the producer's `fact` label. The consent predicate is true when host policy either permits a without a separate prompt or records approval bound to the action digest. Confidence alone never changes authority.

a) Epistemic non-escalation invariant: No execution path may derive action authority solely from a claim or hypothesis. Enforcement occurs at the authorization boundary rather than in model prompting, so the rule remains meaningful when the planner is probabilistic or prompt-injected.

D. Deterministic U0 profile

U0 contains pure operations such as identity, SHA-256, safe-integer sum, JSON projection, concatenation, literal length, literal redaction, literal search, and deduplication. Nodes are local-only and have no filesystem, network, process, model, device, or accelerator capability. Outputs are deterministic. In the current U0 wire vocabulary they carry either the `fact` tag or the legacy `artifact` tag; the conceptual model treats artifact representation separately from epistemic status. Richer profiles may be layered above U0 only if they preserve policy intersection, information monotonicity, and epistemic non-escalation.

E. Canonicalization

Cross-language hashing requires deterministic serialization. The reference profile rejects duplicate object keys, floating-point values, non-finite numbers, unsafe integers, unpaired surrogates, and unknown normative fields. Object keys are ordered lexicographically by Unicode scalar sequence; strings are UTF-8; integers use shortest decimal form; and insignificant whitespace is absent. This follows the objective of JSON canonicalization [28] while intentionally accepting a narrower value domain.

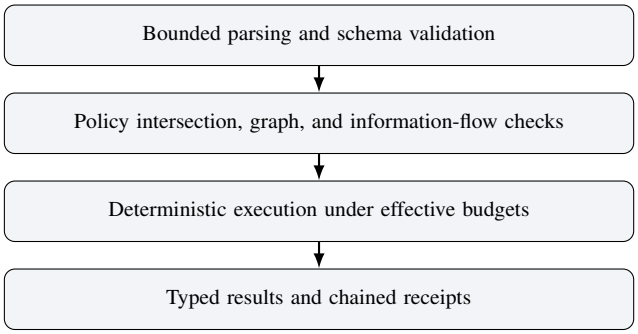


Fig. 2. U0 admission and execution pipeline.

F. Receipt chain

For node n_i, the receipt body contains the manifest digest m, sequence index i, node identifier, operation, requested effect ε_i, placement p_i, canonical input digest, typed output digest, and predecessor r_{i-1}. The receipt is

$$\begin{aligned}r_i = H\big(\mathrm{Canon}(&m, i, n_i, op_i, \varepsilon_i, p_i, \\ &h_i^{\mathrm{in}}, h_i^{\mathrm{out}}, r_{i-1})\big),\end{aligned} \tag{10}$$

with fixed genesis value $r_0 = 0^{256}$. A terminal record binds the final receipt to the export digest.

a) Tamper-detection proposition: Assume that H is collision resistant, canonicalization is deterministic, and the verifier possesses an authentic terminal digest. Any modification, insertion, deletion, or reordering within the complete receipt sequence changes the recomputed terminal digest except with negligible collision probability. The proposition does not establish freshness, prevent rollback to an older complete chain, or attest the platform. Durable replay state, trusted time, signatures, and remote attestation are separate profiles.

G. Binding semantics

Bindings carry the same canonical manifest and structured contract result. Transport success states only that a message exchange completed. AUEC reports a separate state such as accepted, rejected, completed, aborted, or input-required. Transport metadata may negotiate a versioned extension, but it cannot expand host policy.

V. Reference Implementation

A. Runtime structure

The reference implementation is written in Python and separates canonicalization, schema and semantic validation, operation dispatch, idempotency state, receipt verification, and protocol bindings. It neither invokes a shell nor evaluates planner-supplied Python or JavaScript. Operations are selected from a fixed registry and receive immutable, validated inputs.

Admission follows the phases shown in Figure 2. The order is observable: multi-fault inputs must produce stable error families. Parse and schema failures precede graph failures; graph failures precede information-flow and budget failures. Portable error families appear on the wire, while detailed implementation diagnostics remain local.

B. Host policy and operation registry

Host policy declares admitted profiles, operations, effects, placements, classification ceilings, claim-export rules, and budget maxima. The policy is loaded independently of manifest content. Runtime data cannot create authority. For example, text found in a local resource cannot enable network access or register a new operation.

The default U0 policy admits pure local operations only. Any profile that exposes filesystem, process, network, model, accelerator, or device access must provide a separately specified host interface. Naming a capability in a manifest is insufficient to obtain it.

The implementation enforces epistemic non-escalation conservatively in U0: a node may emit only a deterministic value tagged `fact` or, under the legacy U0 vocabulary, `artifact`; every admitted operation declares the `pure` effect. Consequently, U0 contains no transition that turns a claim into action authority. The verification and action-bound consent transitions in Equation 9 are normative requirements for richer profiles; they are not experimentally validated by the U0 conformance study.

C. Persistence and idempotency

A SQLite/WAL store records tenants, idempotency keys, request digests, and completed results. Reusing a key with the same tenant and request returns the recorded result; reusing it with different content is rejected. A single-flight map suppresses concurrent duplicate work within one process. The implementation does not claim distributed exactly-once execution across multiple hosts.

Receipt journals are hash linked and flushed before a terminal state is exposed. Earlier engineering stages include recovery and tamper tests, but journal durability is not the independent variable in the conformance study reported here.

D. MCP binding

The gateway supports the tested stable 2025-11-25 profile and the pinned 2026-07-28 profile. Ordinary MCP tools remain separate from manifest admission. A read-only, zero-argument runtime-information tool is advertised before the strict manifest executor so that a generic conformance fixture can call the first tool without weakening manifest validation. Modern paths implement request metadata, header/body consistency, input-required results, progress streams, and subscriptions.

The gateway does not reinterpret protocol success as execution-contract acceptance. A manifest error remains an AUEC rejection even when the enclosing MCP request was delivered successfully.

E. A2A and HTTP bindings

The A2A gateway exposes agent discovery, JSON-RPC and HTTP+JSON operations, task lifecycle, artifacts, and optional AUEC metadata. Generic A2A messages remain valid without the extension. A transport-neutral HTTP interface supports direct experiments and isolates contract tests from protocol-specific fixtures.

A2A is an incomplete evidence path in this study because the unchanged upstream TCK remains red. Locally patched TCK behavior is not reported as official conformance.

F. Packaging and reproducibility

The artifact contains pinned runner packages and lockfiles, raw conformance outputs, source archives, a wheel, an SBOM, cryptographic hashes, and deterministic ZIP construction. Evidence Bundle EB-2026-08-02 was executed natively on Windows 11. Linux VM or bare-metal and macOS evidence were unavailable at that checkpoint and are therefore reported as external-validity gaps.

VI. Evaluation Methodology

The evaluation asks whether the tested deterministic contract is compatible with existing protocol bindings and whether selected conformance outcomes are causally sensitive to the implementation mechanisms claimed. End-to-end utility and production security are outside the study defined in Section I-C.

A. Use of AI tools in the research workflow

OpenAI ChatGPT and Codex assisted with literature discovery and synthesis, initial drafting, reference-runtime and test generation, debugging, evidence organization, LaTeX preparation, and deterministic artifact automation. Their outputs were treated as candidate material rather than evidence. Protocol counts were taken from sealed execution records; claims were checked against the claim–evidence matrix and cited primary sources. The human author selected the research questions, accepted or rejected proposed changes, and remains responsible for the report.

B. Research questions

- **RQ1 - Binding compatibility:** Can one gateway satisfy unchanged official MCP scenarios for the tested stable, 2026-07-28 release-candidate, and associated draft profiles while preserving strict manifest admission?
- **RQ2 - Causal sensitivity:** Do selected green outcomes become red when the corresponding implementation mechanisms are disabled?
- **RQ3 - Evidence integrity:** Are incompatible or incomplete results retained rather than hidden in expected-failure baselines?
- **RQ4 - Claim boundary:** Which conclusions follow from conformance evidence, and which require independent, production, or provider-level experiments?

C. Evidence-bundle admission

The study uses sealed Evidence Bundle EB-2026-08-02, identified by SHA-256 `e8d07f3108bfaae73b165c663ba2993057d6fc844931f85ef74b3dd5b187c59c`. The bundle contains exact runner packages and lockfiles used to execute official scenarios without local modifications. No expected-failure baseline was configured. Raw standard output, standard error, per-profile summaries, and release decisions are retained.

D. Official MCP profiles

The profiles were executed separately:

- MCP revision 2025-11-25, active scenarios, official runner 0.1.16;
- MCP 2026-07-28 release-candidate profile, active scenarios, official runner 0.2.0-alpha.10;
- the associated tested draft scenario set from the same pinned runner.

The separation prevents a combined count from being interpreted as evidence for a particular revision. Exact scenario registries are archived with the bundle.

E. Causal negative controls

A green suite is weak evidence when the relevant code path is not exercised. Four isolated mutations are therefore applied to disposable SUT copies while preserving each scenario's preconditions: the strict manifest executor is placed first in the tool list; HTTP optional-whitespace removal is disabled; `InputRequiredResult` is disabled; and subscription handling is broken while the capability remains advertised. The targeted scenario must fail for each mutation, after which the unmodified implementation is restored and rerun. These controls test four mechanisms rather than the runner as a whole.

F. A2A preservation test

The pinned upstream A2A TCK is executed unchanged for JSON-RPC and HTTP+JSON. No local patch is labeled official. Failures and skips remain in the report, so unfavorable evidence is not redefined during publication preparation.

G. Metrics and decision rules

For MCP, we record runner identity, profile, scenario count, success, failure, informational checks, and expected-failure configuration. Each negative control records the mutation, targeted scenario, observed failure, and post-restoration result. A2A results retain pass, fail, and skip counts without turning non-applicable tests into evidence of conformance.

RQ1 is supported only when all applicable official MCP checks pass with no expected-failure baseline. RQ2 requires a single-mechanism mutation to turn its target red and restoration to return it to green. RQ3 requires preservation of the red A2A record. RQ4 is answered through the claim–evidence matrix, which distinguishes executed, implementation-supported, proposed, inherited, red, and blocked evidence.

TABLE II
OFFICIAL MCP CONFORMANCE RESULTS IN EVIDENCE BUNDLE EB-2026-08-02.

Profile	Scen.	Pass	Fail	Info
2025-11-25 active	30	39	0	1
2026-07-28 RC profile	20	22	0	1
2026-07-28 tested draft	19	85	0	0

VII. RESULTS

A. Official MCP conformance

Table II reports the unchanged official-runner results. A scenario may emit more than one conformance check, so the check count can exceed the scenario count. All applicable checks pass, no expected-failure baseline is configured, and the SUT standard-error logs are empty.

These counts support RQ1 for the exact runner–profile combinations listed in Section VI. They do not imply compatibility with every MCP implementation, future revision, or optional extension.

B. Causal negative controls

Each isolated mutation made its targeted official scenario fail, and restoration returned the scenario to success (Table III). The results show that tool ordering, optional-whitespace handling, input-required results, and subscription behavior were exercised rather than merely advertised.

The controls support RQ2 for these mechanisms; they do not establish the fault-detection power of the complete runner.

C. A2A results remain incomplete

Table IV preserves the upstream A2A results. JSON-RPC has one failure and HTTP+JSON has two. The high skip count reflects capability and transport conditions in the upstream suite; skipped tests are not counted as conformance evidence.

The engineering record associates the failures with upstream TCK behavior that was publicly under discussion at the evidence date. That diagnosis is not used to redefine the outcome: official A2A conformance remains unsupported. Preserving the unfavorable result supports RQ3.

D. Interpretation

The evidence supports three conclusions. The gateway interoperates with the tested official MCP profiles; the four selected mechanisms causally affect their targeted outcomes; and the artifact separates green transport conformance from unresolved A2A and external-validation gates. Claims outside that boundary remain excluded by Section I-C.

VIII. SECURITY ANALYSIS

A. The planner is an untrusted proposer

The planner never owns host policy. A model-generated manifest, prompt-injected document, or compromised cloud account cannot create authority merely by naming it. A host may expose a bounded resource handle, but the manifest cannot convert that handle into arbitrary path access. Text such as "ignore policy" remains data and has no effect on dispatch.

TABLE III
CAUSAL NEGATIVE CONTROLS. S, F, AND W DENOTE SUCCESS, FAILURE, AND WARNING CHECKS IN THE MUTATED RUN.

Disabled mechanism	Official scenario	Mutated result	Outcome
Safe zero-argument tool ordering	`http-header-validation`	12 S / 1 F	detected
HTTP optional-whitespace removal	`http-header-validation`	12 S / 1 F	detected
`InputRequiredResult`	`input-required-result-basic-elicitation`	0 S / 1 F	detected
Subscription runtime while advertised	`server-stateless`	26 S / 2 F / 2 W	detected

TABLE IV
UNMODIFIED UPSTREAM A2A TCK RESULTS RETAINED AS RED EVIDENCE.

Binding	Pass	Fail	Skip
JSON-RPC	79	1	185
HTTP+JSON	76	2	187

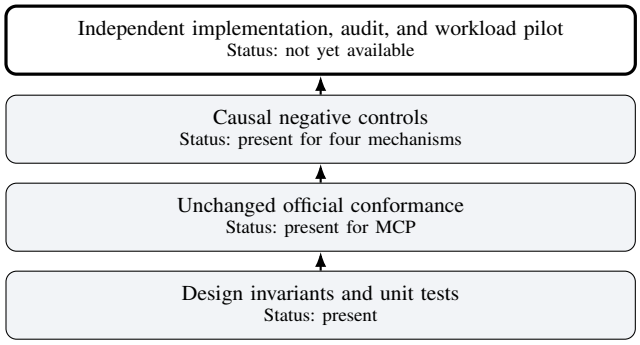


Fig. 3. Evidence hierarchy used to delimit the claims.

B. Parser and graph boundary

Manifests are bounded by byte length, nesting depth, item count, node count, output size, and wall time. The parser rejects duplicate keys, floating-point and non-finite values, unsafe integers, invalid Unicode, and unknown normative fields. Graph validation rejects cycles and unresolved references before execution. These controls reduce ambiguity and amplification; they do not eliminate denial-of-service risks in richer profiles.

C. Capabilities and effects

U0 admits pure operations under local placement and exposes no ambient filesystem, network, process, model, device, or accelerator interface. Capability-bearing profiles require separately specified host interfaces. WebAssembly's embedder-controlled environment is a suitable substrate for portable components, but host imports and embedder configuration remain part of the trusted computing base [3].

D. Information flow and egress

Classification monotonicity prevents explicit downgrading, and secret egress is disallowed in the base profile. These rules do not prove noninterference. Timing, output length, access patterns, error behavior, embeddings, and future model interfaces may leak information. Production profiles require side-channel analysis and explicit, testable declassification procedures.

E. Epistemic and action safety

The rule `claim` $\neq$ authority addresses a recurrent agent hazard: a probabilistic statement can be syntactically valid and still false. It is an authorization rule, not a hallucination detector. U0 enforces the conservative base case by excluding claims and hypotheses from executable outputs and allowing only pure effects. Richer profiles would need an explicit verifier and action-bound authorization path. When human approval is required, consent must bind the exact action digest.

F. Receipt guarantees and limits

Given deterministic canonicalization, collision resistance, and an authentic terminal digest, the receipt chain detects internal modification, insertion, deletion, and reordering of a complete execution record. It does not establish freshness, prevent rollback to an older complete chain, or attest platform integrity. Persistent replay state, signatures, trusted time, and RATS-style appraisal are separate layers [19].

G. Residual risk

The artifact is not a production sandbox. It lacks an external penetration test, an independently developed implementation, independently signed builds, and a production-grade multi-tenant review. A compromised operating system can subvert policy enforcement and receipts. A deterministic operation may also be reproducible yet undesirable when the admission policy is wrong. These limitations preclude production deployment at this stage.

IX. DISCUSSION

A. A contract layer rather than another transport

A practical standards path should extend adopted protocols rather than introduce another network stack. MCP already provides discovery and tool invocation and a formal Specification Enhancement Proposal process [29]; A2A provides agent discovery, messages, tasks, and artifacts. AUEC adds optional execution semantics that can be negotiated through those transports. Offline and browser bindings remain possible without making any binding normative for the contract itself.

B. From the IAScript intuition to an authority contract

The project began with a JavaScript-like intuition: a remote AI service should be able to describe portable work that heterogeneous local user agents understand. That intuition survives as an authoring and interoperability goal, but it is not an adequate security boundary. Sending general-purpose

remote code would reproduce the very authority problem the design is meant to solve.

A future IAScript-style language can therefore serve as a developer-facing syntax that compiles to an AUEC manifest. The host executes only the admitted contract, not the source language's apparent intent. The distinctive contribution is not declarative policy or capability intersection in isolation. It is the placement of epistemic validation at the same boundary that narrows capabilities, placement, egress, budgets, and effects, followed by portable execution evidence. This design addresses a proposer that may be probabilistic, mistaken, or prompt-injected.

C. Adoption incentives

Provider GPU savings are not assumed to be the primary incentive. More immediate motivations are data minimization, offline operation, latency, predictable API expenditure, organizational control, and auditable delegated work. These benefits can be pursued without exposing model internals or modifying the remote model.

D. Performance and quality

Local–cloud partitioning can reduce bandwidth, latency, energy, or cloud-token use, but the optimum depends on workload and may reduce quality [10], [11], [12], [14]. The base profile does not prescribe an optimizer. It supplies hard constraints and evidence around which an optimizer can be evaluated. A provider-authorized study should compare cloud-only, local-preprocessing, and hybrid modes on paired workloads, measuring task quality, latency, tokens, energy, privacy leakage, and erroneous local-routing decisions.

E. Standardization strategy

A staged process is more credible than a single broad proposal. The first release should publish U0 manifests, canonicalization rules, receipts, and authority vectors. Independent implementations should precede any representation freeze. A narrow MCP extension can then negotiate contract metadata, while an A2A extension note preserves generic messages. A WASI component profile should follow only when host capabilities, cancellation, and preemption are testable. Protocol conformance must remain separate from production security certification.

Combining a transport, authoring language, runtime, economic model, and split-inference mechanism in one proposal would make both review and adoption unnecessarily difficult.

F. Artifact publication and licensing

The release separates the open contract from its reference implementation: narrative specifications and documentation use CC BY 4.0, separable interoperability material uses Apache-2.0, and the reference runtime uses AGPL-3.0-only. The specification can therefore be implemented independently without reusing the reciprocal runtime. This distribution choice is not evidence of legal novelty or standards adoption.

X. Threats to Validity

a) Construct validity: Conformance checks measure protocol behavior rather than full security, usability, utility, or semantic correctness. A `fact` denotes a deterministic result under an operation's semantics; it is not a formal assertion of real-world truth.

b) Internal validity: The implementation and most supporting engineering were produced within one program, creating a risk of common-mode assumptions. Unchanged external runners reduce that risk but do not eliminate it. The four negative controls show that their target mechanisms are exercised; they do not establish comprehensive mutation adequacy.

c) External validity: The results apply to the listed MCP runner versions and the Python reference gateway. They do not establish compatibility with all SDKs, operating systems, browsers, or providers. Evidence Bundle EB-2026-08-02 includes native Windows execution; Linux VM or bare-metal and macOS remained unavailable at that checkpoint.

d) Conclusion validity: The MCP and A2A counts are exact artifact observations. No statistical inference is drawn from them, and results from local-routing or split-computing literature are not reinterpreted as measurements of AUEC.

e) Artifact validity: Sealed archives, hashes, lockfiles, and raw outputs improve traceability but cannot show that the development host was uncompromised. The provenance is local and unsigned. Independent builds and transparency logs remain external gates.

f) Standards evolution: MCP, A2A, WASI, and browser-agent interfaces evolve rapidly. A transport-neutral contract reduces coupling, but each binding still requires maintenance as upstream wire shapes and conformance scenarios change.

XI. Reproducibility and Artifact Availability

The release candidate contains three public layers: the report sources, bibliography, figures, and claim–evidence matrix; a repository candidate containing the specification, reference runtime, examples, and public tests; and selected conformance summaries with checksums identifying the underlying forensic record.

Evidence Bundle EB-2026-08-02 has SHA-256 `e8d07f3108bfaae73b165c663ba2993057d6fc844931f85ef74b3dd5b187c59c`. The repository candidate includes the official MCP summary, negative-control report, A2A report, operating-system matrix, and machine-readable validation summary. Public smoke tests exercise successful execution and receipt verification, secret-egress rejection, duplicate-key rejection, canonical floating-point rejection, and the pure local default policy.

Release coordinates are omitted from this private review build and are injected only after separate GitHub and Zenodo publication authorization.

A. Reproduction commands

From the repository root:

```
make test
make technical-report
make audit
```

The prepared CI workflow executes the tests and report build on a clean Linux runner. The claim–evidence matrix labels each assertion as executed, implementation-supported, proposed, inherited, red, or blocked.

XII. Conclusion

AUEC defines a provider-neutral contract layer for hybrid AI execution. A remote planner proposes work, and a host-controlled user agent derives the effective contract. The manifest joins operations, data classifications, epistemic status, representation, placement, effects, budgets, and exports. Its deterministic U0 profile provides a small interoperable base; canonical hash-linked receipts make changes to a complete execution history detectable under explicit assumptions.

The reference gateway passes the tested official MCP profiles, and four causal negative controls demonstrate sensitivity of selected scenarios to their corresponding mechanisms. Incomplete A2A results remain in the record. Together, these results support a pre-standard systems artifact and a narrow extension proposal within the scope defined in Section I-C.

The decisive next evidence must come from outside the originating implementation: independently developed runtimes, unchanged upstream A2A results after the relevant TCK questions are resolved, independently signed builds, external security review, and provider-authorized workload trials. The contract will be useful only if independent hosts enforce the same narrow semantics and reject the same unsafe proposals.

Acknowledgment

AIEW/AUEC was conceived and directed by Mohammed Messaoudene. OpenAI ChatGPT and Codex assisted with research synthesis, drafting, code, tests, LaTeX, and artifact automation. The author remains solely responsible for the claims, evidence, citations, code, and wording; the tools are not authors, sponsors, partners, or endorsers [30].

References

[1] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[2] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, D. Gohman, L. Wagner, A. Zakai, J. Bastien, and M. Holman, "Bringing the web up to speed with webassembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. New York, NY, USA: ACM, 2017, pp. 185–200.

[3] W3C WebAssembly Working Group, "Webassembly core specification," W3C Candidate Recommendation Draft, 2026, accessed 3 August 2026. [Online]. Available: https://www.w3.org/TR/wasm-core/

[4] B. Hayes and Y. Wuyts, "WASI 0.3 launched," Bytecode Alliance, 2026, 11 June 2026; accessed 3 August 2026. [Online]. Available: https://bytecodealliance.org/articles/WASI-0.3

[5] Model Context Protocol Contributors, "Model context protocol specification, revision 2025-11-25," 2025, accessed 3 August 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25

[6] ——, "Model context protocol 2026-07-28 release candidate," 2026, pre-release and draft; accessed 3 August 2026. [Online]. Available: https://github.com/modelcontextprotocol/modelcontextprotocol/releases

[7] ——, "MCP conformance test suite," 2026, exact runner versions archived with the artifact. [Online]. Available: https://github.com/modelcontextprotocol/conformance

[8] Agent2Agent Project, "Agent2agent protocol specification, version 1.0," 2026, linux Foundation project; exact version cited; accessed 3 August 2026. [Online]. Available: https://github.com/a2aproject/A2A/releases/tag/v1.0.0

[9] ——, "Agent2agent protocol technology compatibility kit," 2026, exact upstream commit and red results archived with the artifact. [Online]. Available: https://github.com/a2aproject/a2a-tck

[10] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems*. New York, NY, USA: ACM, 2017, pp. 615–629.

[11] E. Li, Z. Zhou, and X. Chen, "Edge intelligence: On-demand deep learning model co-inference with device-edge synergy," in *Proceedings of the 2018 Workshop on Mobile Edge Communications*. New York, NY, USA: ACM, 2018, pp. 31–36.

[12] Y. Matsubara, M. Levorato, and F. Restuccia, "Split computing and early exiting for deep learning applications: Survey and research challenges," *ACM Computing Surveys*, vol. 55, no. 5, 2022.

[13] J. Shao and J. Zhang, "Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems," 2019, preprint.

[14] J. O. Agyemang, J. J. Kponyo, E. Amponsah, G. M. A. Boakye, and K. O.-B. O. Agyekum, "Local-splitter: A measurement study of seven tactics for reducing cloud LLM token usage on coding-agent workloads," 2026, preprint; not treated as peer-reviewed evidence in this paper.

[15] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[16] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.

[17] T. Lebo, S. Sahoo, D. McGuinness, K. Belhajjame, J. Cheney, D. Corsar, D. Garijo, S. Soiland-Reyes, S. Zednik, and J. Zhao, "PROV-O: The PROV ontology," W3C Recommendation, 2013, 30 April 2013. [Online]. Available: https://www.w3.org/TR/prov-o/

[18] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," in *Advances in Cryptology – CRYPTO '90*, ser. Lecture Notes in Computer Science, vol. 537. Berlin, Germany: Springer, 1991, pp. 437–455.

[19] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote attestation procedures (RATS) architecture," RFC Editor, Tech. Rep. RFC 9334, Jan. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9334.html

[20] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *28th USENIX Security Symposium*. Santa Clara, CA, USA: USENIX Association, 2019, pp. 1393–1410. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[21] A. Basu, "Tool receipts, not zero-knowledge proofs: Practical hallucination detection for ai agents," 2026, preprint; 9 March 2026.

[22] Q. Ye and J. Tan, "Agent contracts: A formal framework for resource-bounded autonomous ai systems," 2026, arXiv preprint; current version notes workshop acceptance.

[23] A. Mishra and K. Sharad, "Observability for delegated execution in agentic ai systems," 2026, preprint; 8 June 2026.

[24] Z. Zhou, "Governing dynamic capabilities: Cryptographic binding and reproducibility verification for ai agent tool use," 2026, preprint; 15 March 2026.

[25] J. Figuera, "Notarized agents: Receiver-attested confidential receipts for ai agent actions," 2026, preprint; 2 June 2026.

[26] S. Bradner, "Key words for use in RFCs to indicate requirement levels," RFC Editor, Tech. Rep. RFC 2119, 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2119.html

[27] B. Leiba, "RFC 2119 key words: Clarifying the use of capitalization," RFC Editor, Tech. Rep. RFC 8174, 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8174.html

[28] A. Rundgren, B. Jordan, and S. Erdtman, "JSON canonicalization scheme (JCS)," RFC Editor, Tech. Rep. RFC 8785, Jun. 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8785.html

[29] Model Context Protocol Contributors, "SEP-001: Model context protocol governance," 2025, final process proposal establishing the Specification Enhancement Proposal mechanism; accessed 3 August 2026. [Online]. Available: https://github.com/modelcontextprotocol/modelcontextprotocol/issues/932

[30] Association for Computing Machinery, "Acm publications policy on authorship and generative ai," 2026, accessed 3 August 2026. [Online]. Available: https://www.acm.org/publications/policies/new-acm-policy-on-authorship